UNIVERSITÉ ABDERRAHMANE MIRA – BEJAIA

Faculté des Sciences Exactes

Département d'Informatique

Rapport Mini Projet - Génie Logiciel

Niveau : 3ème Année Ingénieur

Sujet N°3 : Gestion des Affaires Juridiques

Encadrement :

-Chargée de cours : L. Hamza

-Chargées de TP : N. Bair & S. Boussoufa

Réalisé par :

-MAMERI Yousra

-YESSAD Zineddine

Année universitaire 2025 - 2026

Sommaire

Rapport Mini Projet - Génie Logiciel 1
Sommaire 1
1. Introduction 2
1.1 Contexte 2
1.2 Problématique 2
2. Déroulement du Projet 3
2.1 Conception 3
2.2 Cycle de vie du logiciel 3
3. Cycle de vie et étapes de développement du système de gestion juridique 4
3.1 Analyse des besoins 4
3.1.1 Analyse du réel (Étude de l'existant) 4
3.1.2 Description du domaine 6
3.1.3 Qualité exigée du logiciel 6
3.2 Installation et Test Système 7
3.3 Spécifications 8
3.3.1 Les besoins fonctionnels 8
3.3.2 Les besoins non fonctionnels 10
3.4 Test d'acceptation 12
3.5 Conception architecturale 12
3.5.1 Objectif 12
3.5.2 Composants principaux et leurs interactions 12
3.5.3 Contraintes architecturales 12
3.6 Intégration et Test d'intégration 12
3.7 Conception détaillée 13
3.7.1 Conception UML 13
3.7.2 Diagramme de cas d'utilisation 13
3.7.3 Diagramme de classes 16
3.7.4 Le modele Relationnel : 17
3.7.4 Diagrammes de séquence : 18
3.8 Programmation 23

Backend (Spring Boot) 23
Frontend (React / JSX) 23
Base de données (MySQL – WAMP) 24
3.9 Test unitaire 24
4. Le script SQL de la Base De Données: 24
5. Aperçu graphique de l'interface 29
6. Conclusion 31

1. Introduction

Dans le cadre du module de Génie Logiciel, nous avons été amenés à réaliser un mini projet portant sur la conception et la mise en œuvre d'un système de gestion des affaires juridiques. Ce projet a pour objectif de mettre en pratique les notions étudiées en cours, notamment l'analyse des besoins, la modélisation UML, la conception de bases de données et le développement d'applications logicielles.

1.1 Contexte

La gestion des affaires juridiques implique le suivi de nombreux clients, dossiers et séances.

Les méthodes de gestion manuelle peuvent s'avérer inefficaces, chronophages et sujettes à des erreurs, notamment lors de la recherche ou de la mise à jour des informations. La mise en place d'un système informatique dédié permet de centraliser, organiser et automatiser la gestion de ces données, tout en améliorant la fiabilité, la rapidité d'accès et la productivité du travail des utilisateurs.

1.2 Problématique

Dans un environnement juridique, il est souvent difficile de gérer simultanément plusieurs clients et leurs affaires. Les informations éparpillées ou mal enregistrées peuvent entraîner des retards, des pertes de données ou des erreurs dans le traitement des dossiers.
Il devient donc essentiel de concevoir une solution permettant de :

-Assurer le suivi précis de chaque client et de ses coordonnées

-Gérer les affaires juridiques associées à chaque client

-Planifier, enregistrer et consulter les différentes séances liées à ces affaires

2. Déroulement du Projet

2.1 Conception

Cette phase, composée de plusieurs séances de travail, permet de suivre toutes les étapes du cycle de vie de l'application.
Elle inclut l'analyse détaillée du cahier des charges, des besoins exprimés par le client, ainsi que la validation de l'architecture, de l'ergonomie, des algorithmes et des fonctionnalités prévues.
L'objectif est de s'assurer que le système répond correctement aux attentes et qu'il est prêt pour la phase de conception et de développement.

2.2 Cycle de vie du logiciel

Choix du modèle de développement :

Pour la réalisation de notre mini projet intitulé « Système de gestion des affaires juridiques », nous avons choisi d'adopter le modèle en V comme modèle de développement logiciel.

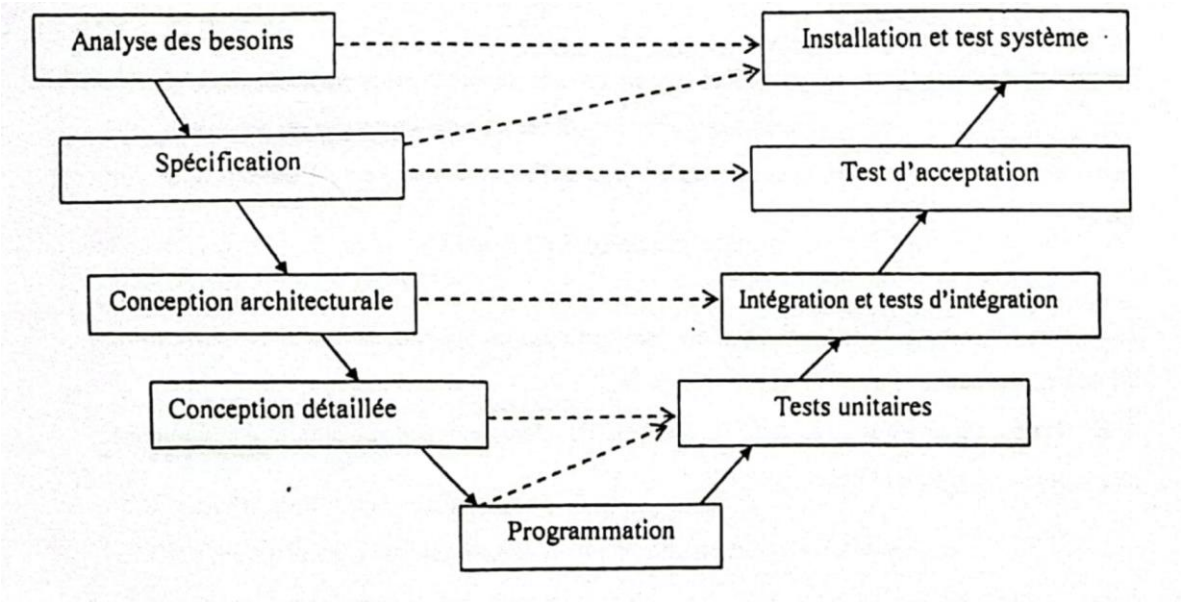

Figure 1 : Modèle en V.

-Ce choix s'explique par plusieurs raisons :

-Le sujet du projet est bien défini dès le départ, avec des besoins clairs et peu susceptibles de changer au cours du développement.

-Le modèle en V est structuré et méthodique, ce qui permet de suivre une progression logique depuis l'analyse jusqu'aux tests.

-Il offre une traçabilité entre les différentes phases : chaque étape de conception (analyse, conception, implémentation) est directement reliée à une phase de test correspondante.

-Ce modèle est particulièrement adapté aux projets académiques où la clarté de la démarche et la qualité de la documentation sont essentielles.

3. Cycle de vie et étapes de développement du système de gestion juridique

3.1 Analyse des besoins

3.1.1 Analyse du réel (Étude de l'existant)

Historique et contexte

Avant l'utilisation d'un système informatique, la gestion des affaires juridiques se faisait essentiellement manuellement ou à l'aide de tableurs basiques. Chaque client possédait un dossier papier contenant ses informations personnelles, les affaires qu'il suivait et les séances associées.

Cette méthode présente plusieurs limites :

-Risque de perte ou de détérioration des dossiers.

-Accès aux informations lent et laborieux.

-Difficulté à suivre plusieurs affaires simultanément.

-Erreurs fréquentes lors de la saisie et du suivi des dates importantes (audiences, jugements).

Problématiques rencontrées

La gestion manuelle (ou semi-informatisée) des affaires juridiques entraîne plusieurs difficultés :

-Suivi des clients : Les informations peuvent être dispersées, mal mises à jour ou incomplètes.

-Gestion des affaires : Impossible de lier efficacement un client à ses multiples affaires.

-Planification des séances : Les dates d'audience ou de réunion peuvent se chevaucher ou être oubliées.

-Recherche et reporting : Retrouver rapidement un dossier ou générer un rapport est long et sujet aux erreurs.

Systèmes existants

Certains cabinets utilisent déjà des logiciels généralistes (type tableur ou logiciel de gestion de documents), mais ces solutions présentent des limites :

-Peu adaptés au domaine juridique.

-Manquent de fonctionnalités pour lier clients, affaires et séances.

-Interface peu ergonomique et pas centralisée.

3.1.2 Description du domaine

Le système concerne la gestion des affaires juridiques dans un environnement où les clients sont suivis pour différentes affaires auprès de tribunaux.

Le domaine comprend plusieurs éléments clés :

Clients : Chaque client possède un identifiant, un nom, un prénom, une date de naissance, une wilaya et un numéro de téléphone. Un client peut être lié à plusieurs affaires juridiques.

Affaires : Chaque affaire a un numéro unique, un thème, une nature, un tribunal, une date de début et une date de jugement. Une affaire peut être associée à plusieurs séances.

Séances : Ce sont des audiences ou réunions au tribunal, caractérisées par un identifiant, une date et une heure de début.

Acteurs du système :

-Client : bénéficiaire du suivi des affaires

-**Avocat / Secrétaire** : utilisateur principal qui gère les clients et les affaires

-**Administrateur** : responsable de la configuration et de la maintenance du système

3.1.3 Qualité exigée du logiciel

Pour garantir le bon fonctionnement et la pérennité de l'application de gestion des affaires juridiques, il est essentiel que le logiciel respecte plusieurs critères de qualité adaptés aux besoins spécifiques du domaine juridique.

-Fiabilité : L'application doit fonctionner sans interruption, même lors de fortes sollicitations ou en cas d'erreurs utilisateur. Par exemple, la gestion des dossiers juridiques ne doit jamais entraîner de perte de données, et les accès aux informations sensibles doivent toujours être disponibles.

-Maintenabilité : Le logiciel doit pouvoir être facilement mis à jour pour intégrer de nouvelles fonctionnalités, corriger des bugs ou s'adapter aux évolutions légales et réglementaires sans perturber le service. Cela est crucial pour assurer la conformité continue aux exigences juridiques.

-Efficacité : L'application doit utiliser de manière optimale les ressources du serveur et du réseau, pour garantir une réponse rapide lors des consultations de dossiers, des recherches clients ou des plannings de séances, même avec un grand volume de données.

-Facilité d'emploi : Les utilisateurs, qu'ils soient administrateurs, juges, avocats ou clients, doivent pouvoir naviguer facilement dans l'application. Par exemple, l'interface doit être intuitive, avec des menus clairs et une gestion simple des notifications pour les audiences à venir.

-Validité : Le logiciel doit répondre précisément aux besoins métier : gestion complète des clients, des affaires, des séances et des salles, avec des droits d'accès stricts selon les rôles attribués. Chaque fonctionnalité doit correspondre aux spécifications du cahier des charges.

-Extensibilité : L'application doit pouvoir être étendue, par exemple en ajoutant de nouveaux modules pour la gestion des documents juridiques ou l'intégration avec des bases de données externes, sans nécessiter une refonte complète du système.

-Réutilisabilité : Certains composants du logiciel, comme les modules d'authentification ou de gestion des utilisateurs, doivent pouvoir être réutilisés dans d'autres applications internes, pour faciliter le développement futur.

-Compatibilité : L'application doit fonctionner correctement sur différents navigateurs web et systèmes d'exploitation, afin d'assurer un accès universel aux utilisateurs, quel que soit leur matériel.

-Portabilité : Le logiciel doit pouvoir être déployé facilement sur différents serveurs ou environnements cloud, facilitant ainsi la montée en charge ou la migration vers des infrastructures plus performantes.

-Intégrité : La protection des données personnelles et juridiques est primordiale. L'application doit garantir que seuls les utilisateurs autorisés peuvent accéder, modifier ou supprimer les informations, avec une traçabilité complète des actions réalisées.

3.2 Installation et Test Système

On procédera ici à l'installation complète de l'application dans son environnement réel afin de vérifier son bon fonctionnement après intégration de tous les modules. Le but est de s'assurer que le déploiement se déroule correctement, que toutes les composantes nécessaires sont opérationnelles et que le système fonctionne de manière stable et cohérente.

Ainsi, on pourra détecter d'éventuels problèmes liés à la configuration, à la compatibilité ou à l'interaction entre les différents modules, garantissant ainsi une mise en service fiable de l'application.

3.3 Spécifications

3.3.1 Les besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités que le système doit offrir aux utilisateurs.

L'application de gestion des affaires juridiques a pour objectif d'assurer une gestion efficace des clients, des affaires, des séances, des salles et des utilisateurs, tout en garantissant un accès sécurisé selon les rôles attribués.

Authentification et gestion des rôles

Le système doit intégrer un mécanisme d'authentification et de gestion des droits :

-Connexion sécurisée pour tous les utilisateurs.

-Rôles distincts : Administrateur, Juge, avocat et Secrétaire&.

-Gestion des sessions et déconnexion automatique après inactivité.

-Possibilité d'ajouter ou modifier un compte utilisateur (pour l'administrateur).

-Gestion des droits d'accès selon le rôle de l'utilisateur (accès complet ou partiel).

Gestion des clients

L'application doit permettre de gérer toutes les informations relatives aux clients :

-Ajouter un client : (Nom, prénom, date de naissance, wilaya, téléphone...).

-Rechercher un client par id, nom ou prénom.

-Lister tous les clients enregistrés.

-Lier chaque client à ses affaires correspondantes.

Gestion des affaires

Chaque affaire est liée à un client et contient des informations juridiques spécifiques. Le système doit permettre de :

-Créer une affaire : (numéro, thème, nature, tribunal, date jugement...).

-Modifier ou consulter une affaire.

-Lister les affaires d'un client donné.

-Filtrer les affaires par tribunal, nature ou période.

-Suivre l'évolution de chaque affaire (en cours, jugée, archivée).

-Consulter l'historique des affaires traitées.

Gestion des séances

Une affaire peut comporter plusieurs séances. Le système doit permettre de :

-Ajouter une séance : (id séance, date, heure début, type, lieu...).

-Modifier ou consulter une séance (sauf si la date du jugement est passée).

-Lister les séances par affaire, tribunal ou date.

-Vérifier le planning des séances et éviter les chevauchements.

Gestion des salles

La gestion des salles d'audience est essentielle pour une planification efficace :

-Vérifier la disponibilité des salles.

-Réserver une salle pour une séance donnée.

-Modifier la disponibilité d'une salle.

-Consulter la liste des salles disponibles ou occupées.

Gestion des utilisateurs

-Création et modification des comptes (par l'administrateur).

-Consultation des informations de chaque utilisateur.

-Attribution des rôles (admin, juge, avocat, secrétaire, client).

Gestion des avocats et du personnel

-Enregistrer les avocats et juges dans le système.

-Associer un avocat à une ou plusieurs affaires.

-Permettre aux juges de consulter les séances liées à leurs affaires.

-Permettre à la secrétaire de planifier et modifier les séances.

Suivi et consultation (Client)

Le client dispose d'un espace personnel pour suivre ses affaires :

-Consulter ses affaires et leurs détails.

-Consulter les séances prévues ou passées.

-Suivre l'évolution de ses jugements (dates, résultats, tribunaux).

Fonctions transverses :

Fonctionnalités générales pour améliorer l'expérience utilisateur :

-Import / Export (PDF).

-Rapports automatiques : calendrier des séances, affaires en attente, statistiques simples.

-Recherche multicritères (thème, tribunal, client, période).

-Tri des tableaux, compteur d'affaires par client.

-Gestion des avocats.

-Archivage automatique des affaires jugées depuis 6 mois.

3.3.2 Les besoins non fonctionnels

Ergonomie et interface :

-Interface simple, intuitive et homogène.

-Navigation fluide entre modules (clients, affaires, séances).

-Tableaux interactifs avec tri et filtrage.

Performance :

-Réponse rapide (moins de 2 secondes pour les opérations courantes).

-Support d'un grand volume de données.

Sécurité :

-Authentification sécurisée (admin, employé).

-Mots de passe chiffrés.

-Protection et confidentialité des données.

Fiabilité :

-Gestion d'erreurs sans interruption.

-Sauvegarde et restauration automatiques.

Portabilité et maintenabilité :

-Compatible Windows / Linux, Mac OS.

-Code clair, modulaire et facilement extensible.

Délai de réalisation :

Le projet doit être développé dans un délai de 6 semaines, comprenant :

-Étude du réel et analyse des besoins

-Conception UML et base de données

-Développement et intégration

-Tests finaux et rédaction du rapport

3.4 Test d'acceptation

On vérifiera ici que l'application répond bien aux besoins exprimés par l'utilisateur. Le but est de s'assurer que le système fonctionne conformément aux exigences fonctionnelles définies au départ et qu'il peut être validé comme produit final. Ainsi, on saura si l'application est prête à être acceptée et mise en service.

3.5 Conception architecturale

3.5.1 Objectif

La conception architecturale a pour but de définir l'organisation globale du système, ses composants majeurs et leurs interactions, afin d'assurer modularité, sécurité, performance et maintenabilité.

3.5.2 Composants principaux et leurs interactions

-**Module Authentification et gestion des rôles :** Toutes les couches pour contrôler l'accès.

-Module Gestion des clients : Base de données clients, interface utilisateur.

-Module Gestion des affaires : Base de données affaires, logique métier pour suivi et filtrage.

-**Module Gestion des séances et salles** : Interface planning, base de données.

-Module Rapports et statistiques : Logique métier pour calculs, export PDF, historique.

-Module Suivi client : Interface client, logique métier, notifications.

3.5.3 Contraintes architecturales

-Sécurité : cryptage des mots de passe, gestion des sessions.

-Performance : accès rapide aux informations critiques (séances, planning).

-Maintenabilité : modularité pour ajouter de nouvelles fonctionnalités (ex : notifications).

-Compatibilité : multi-utilisateurs et rôles différents.

3.6 Intégration et Test d'intégration

On effectuera ici l'assemblage progressif des différents modules développés afin de former un système complet et cohérent. Le but est de vérifier la bonne communication et l'interopérabilité entre les modules, tout en détectant d'éventuels problèmes d'échange de données ou de dépendance.
Ainsi, on s'assure que l'intégration de chaque partie du système se fait correctement et que l'application fonctionne de manière harmonieuse avant de passer aux tests système.

3.7 Conception détaillée

3.7.1 Conception UML

La conception UML constitue une étape essentielle dans le développement du système. Elle permet de représenter de manière graphique et structurée l'organisation du système ainsi que les interactions entre les différents acteurs et composants.
L'objectif est de transformer les besoins fonctionnels analysés précédemment en un modèle conceptuel clair, cohérent et facilement implémentable.

3.7.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation décrit les principales fonctionnalités offertes par le système ainsi que les acteurs qui interagissent avec lui.
Il permet de représenter, de manière synthétique, les différents rôles et les interactions entre les utilisateurs et le système.
Ce diagramme est illustré ci-dessous :

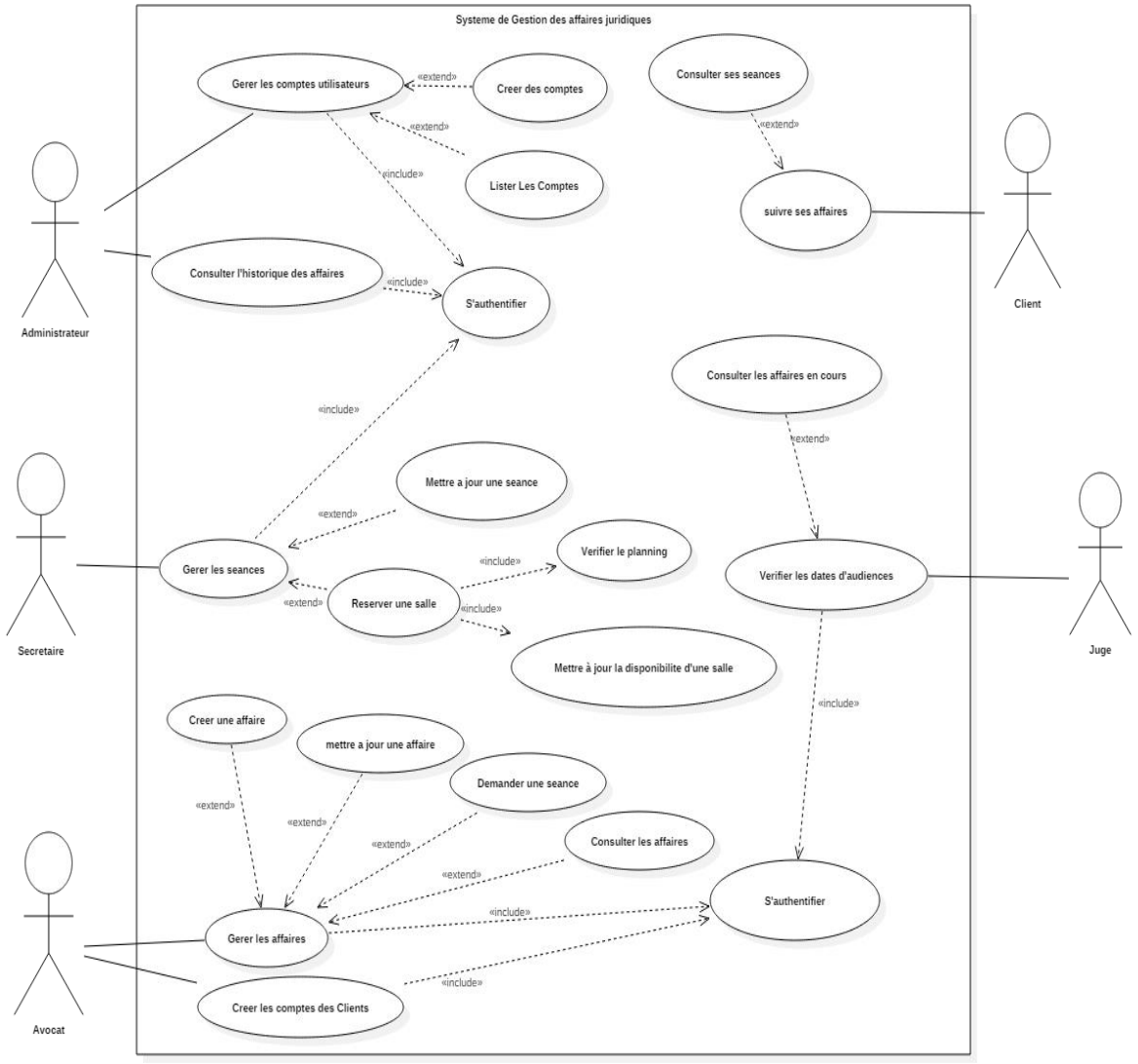

Figure 2 : Diagramme de cas d'utilisation des

Gestion des affaires juridiques.

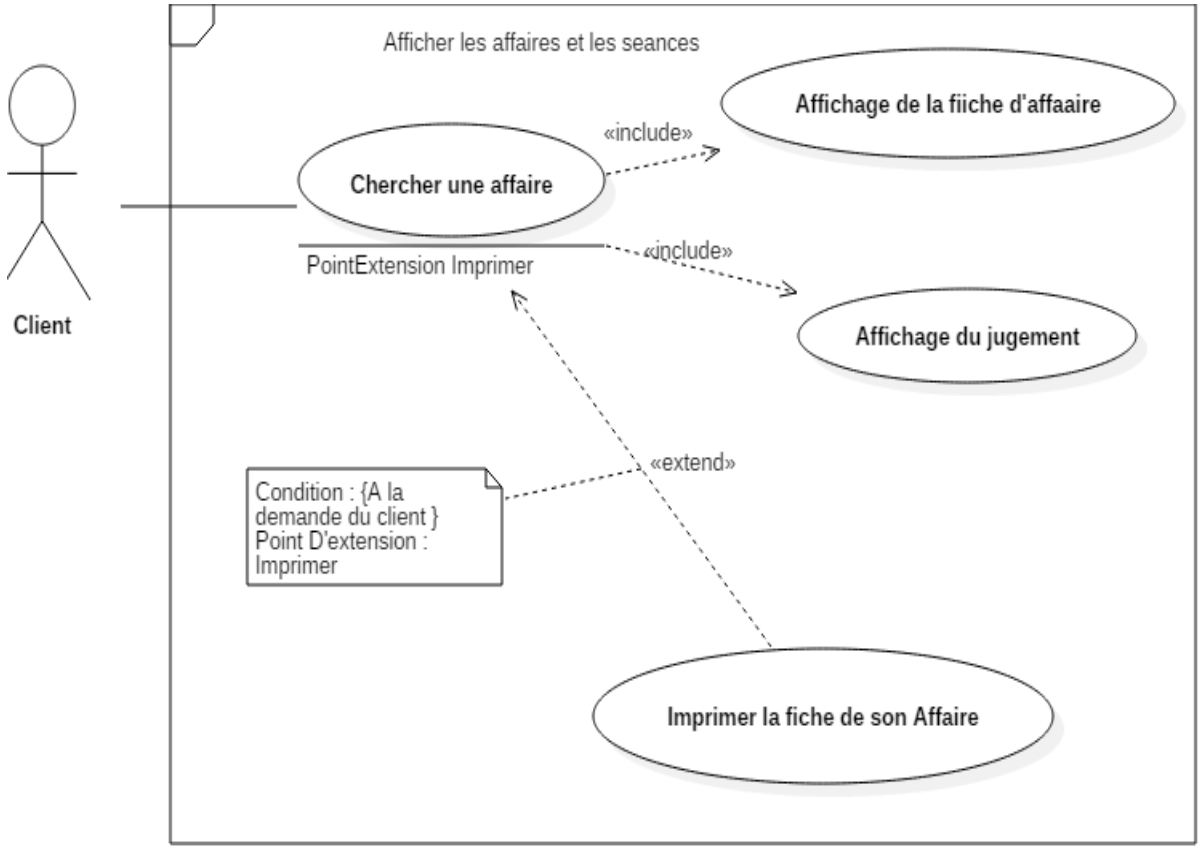

Figure 3 : Diagramme de cas d'utilisation
"Afficher les affaires et les séances"

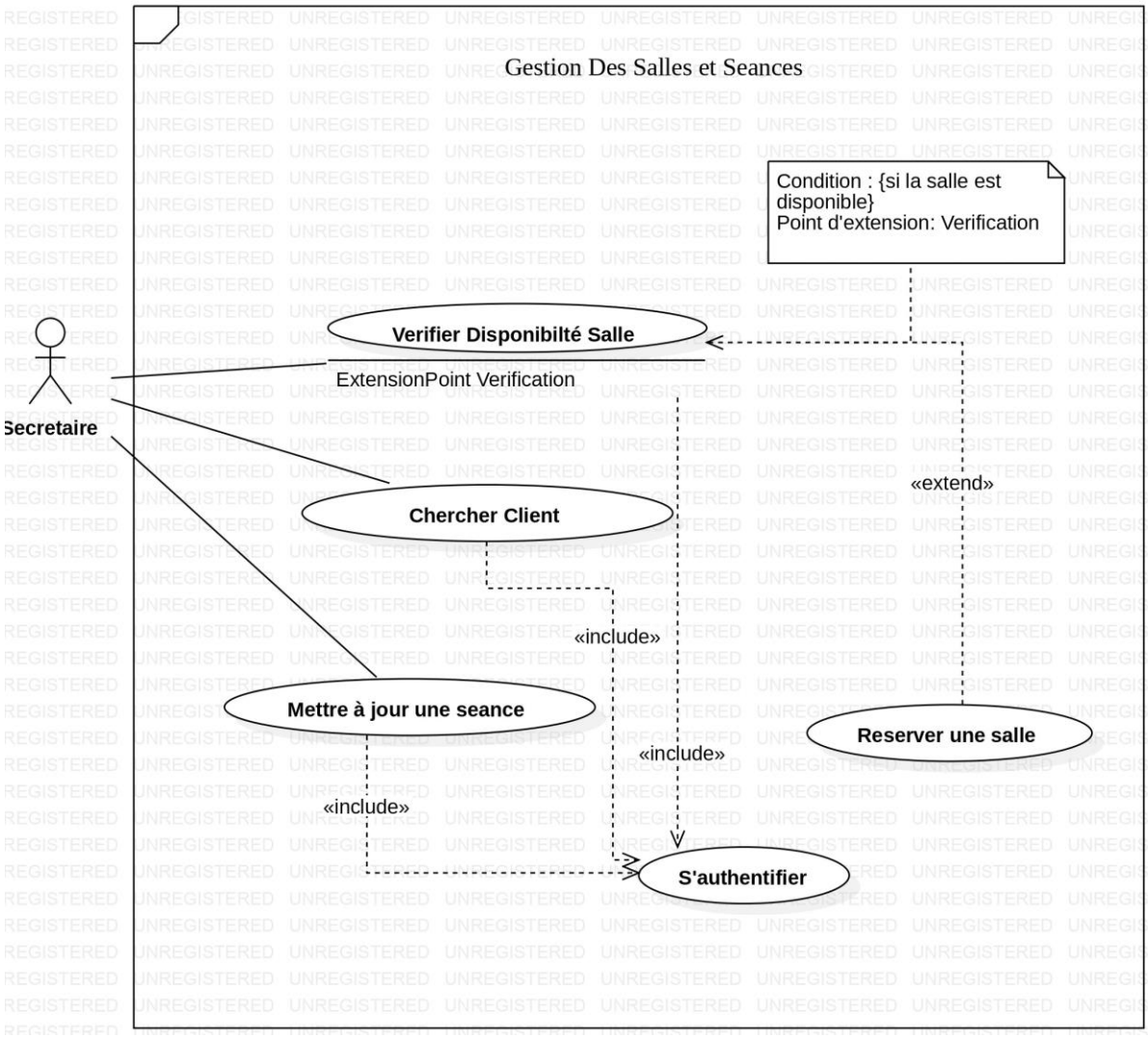

Figure 4 : Diagramme de cas d'utilisation
"Gestion des salles et séances"

3.7.3 Diagramme de classes

Le diagramme de classes permet de modéliser la structure interne du système en identifiant les entités, leurs attributs et les relations entre elles.

Ce diagramme est illustré ci-dessous :

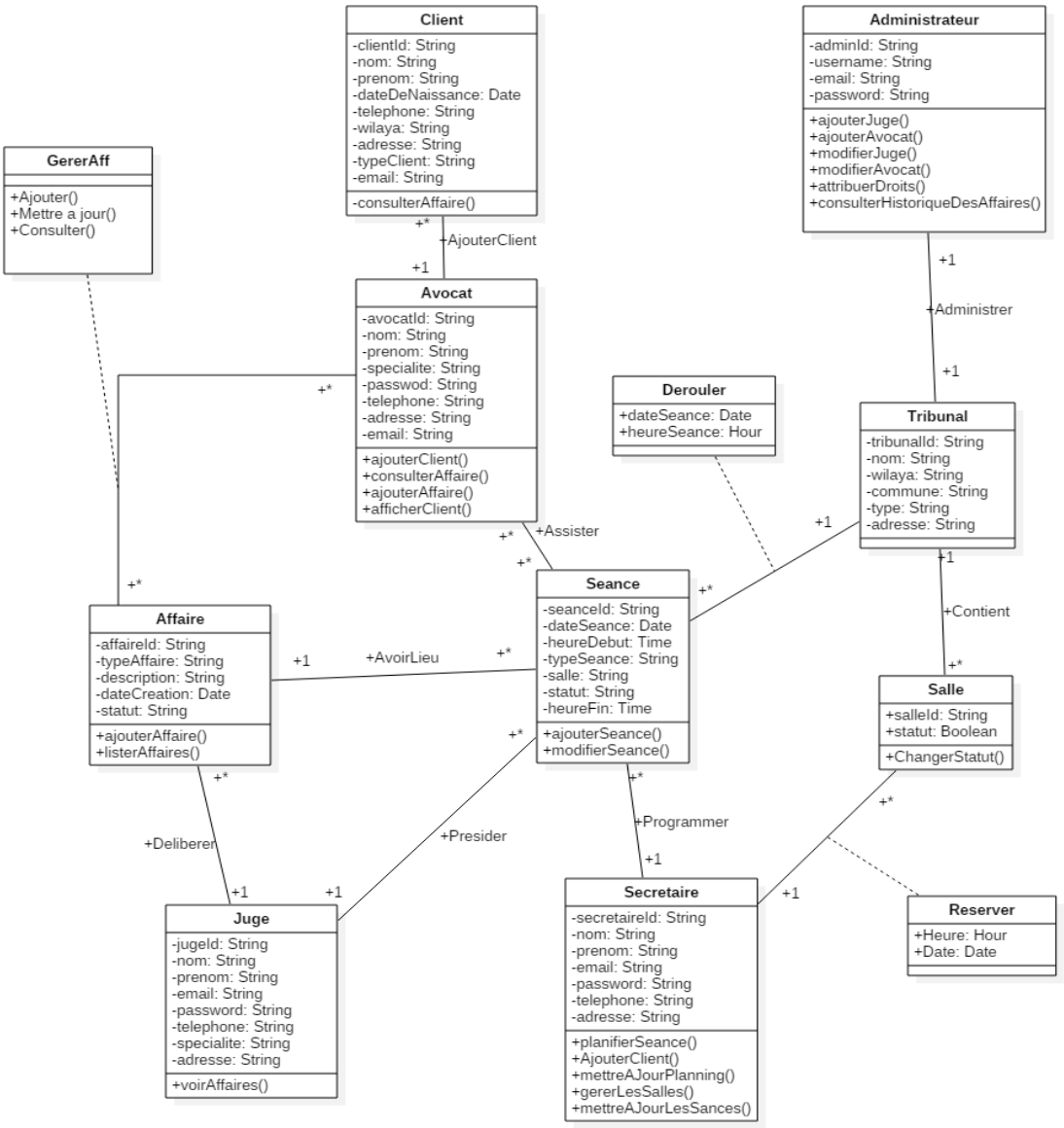

Figure 5 : Diagramme de classes de

Gestion des affaires juridiques.

3.7.4 Le modele Relationnel :

-TRIBUNAL (tribunalId, nom, willaya, commune, adresse, type, # adminId).

-SALLE (salleId, statut,#tribunalId, #secretaireId), Règle de passage numéro 1.

-AVOCAT (avocatId, nom, prenom, email, password, telephone, adresse, specialite), Règle de passage numéro 2.

-CLIENT (clientId, nom, prenom, email, telephone, adresse, typeClient, dateDeNaissance, willaya,#avocatId), Règle de passage numéro 1.

-ADMIN (adminId, username, email, password).

-SECRÉTAIRE (secretaireId, nom, prenom, email, password, telephone).

-JUGE (jugeId, password, nom, prenom, email, telephone, specialite).

-SÉANCE (seanceId, dateSeance, heureDebut, heureFin, statut,#affaireId, #secretaireId, #tribunalId, #jugeId), Règle de passage numéro 1.

-GERERAFFAIRE (#affaireId, #avocatId), Règle de passage numéro 2.

-AFFAIRE (affaireId, typeAffaire, description, dateCreation, statut, #jugeId), Règle de passage numéro 1.

3.7.4 Diagrammes de séquence :

Les diagrammes de séquence décrivent le déroulement dynamique des interactions entre les objets du système lors d'un scénario donné.

Ce diagramme est illustré ci-dessous :

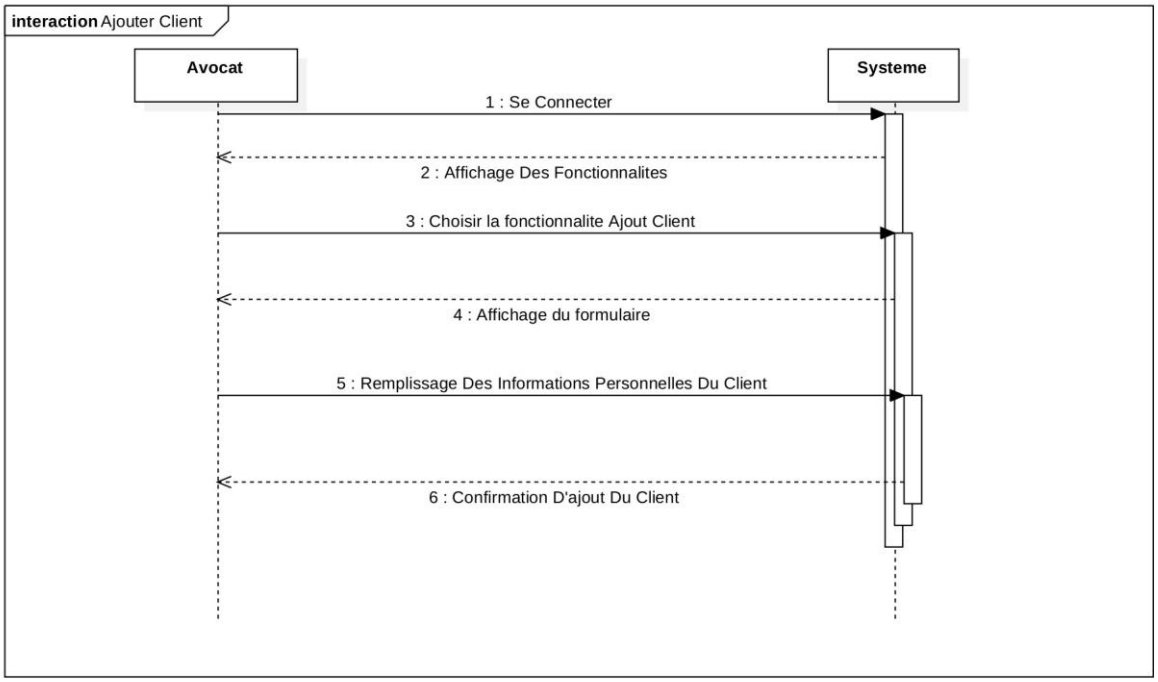

Figure 6 : Diagramme de séquence
“Ajouter Client”

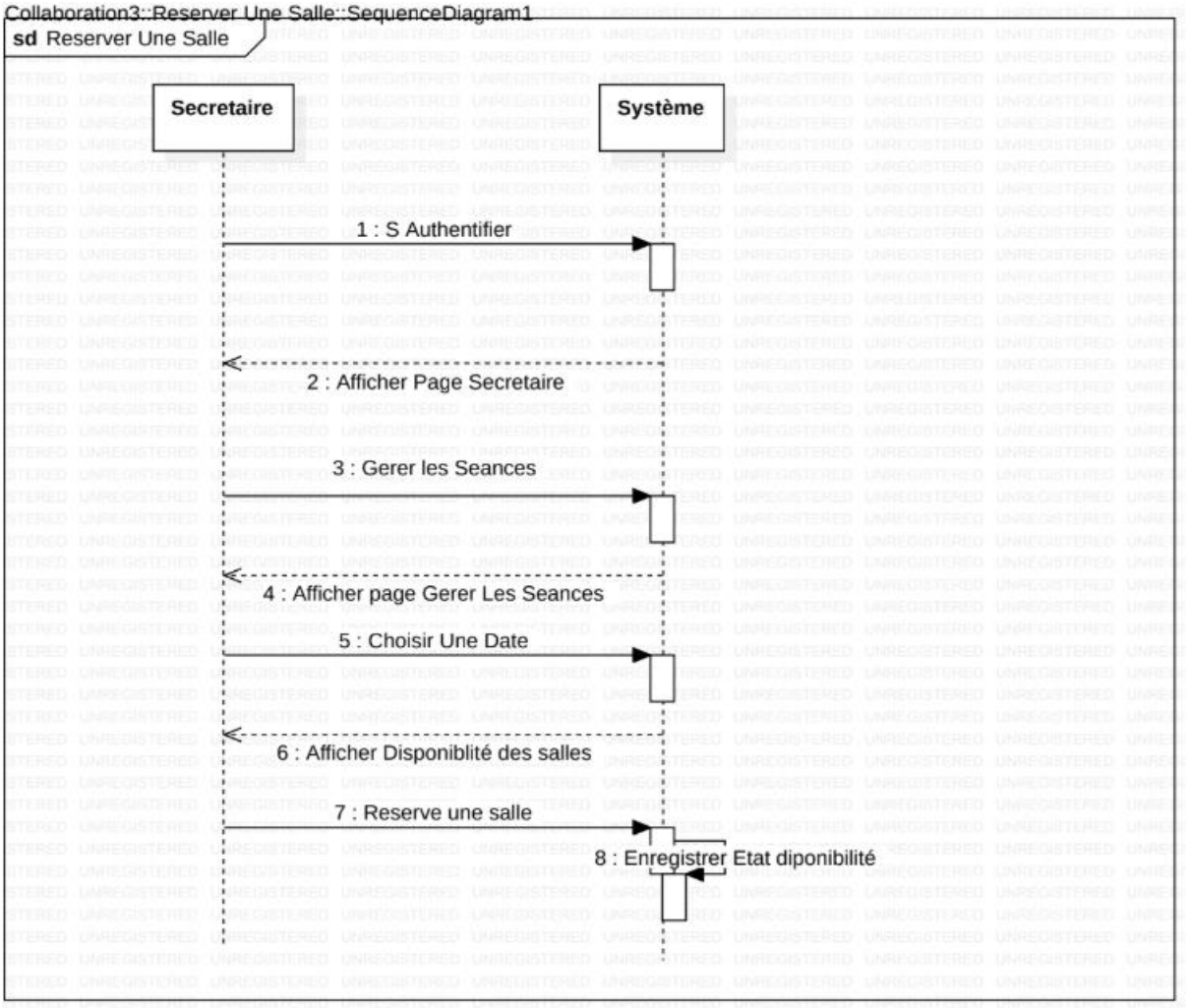

Figure 7 : Diagramme de séquence
"Réserver Une Salle"

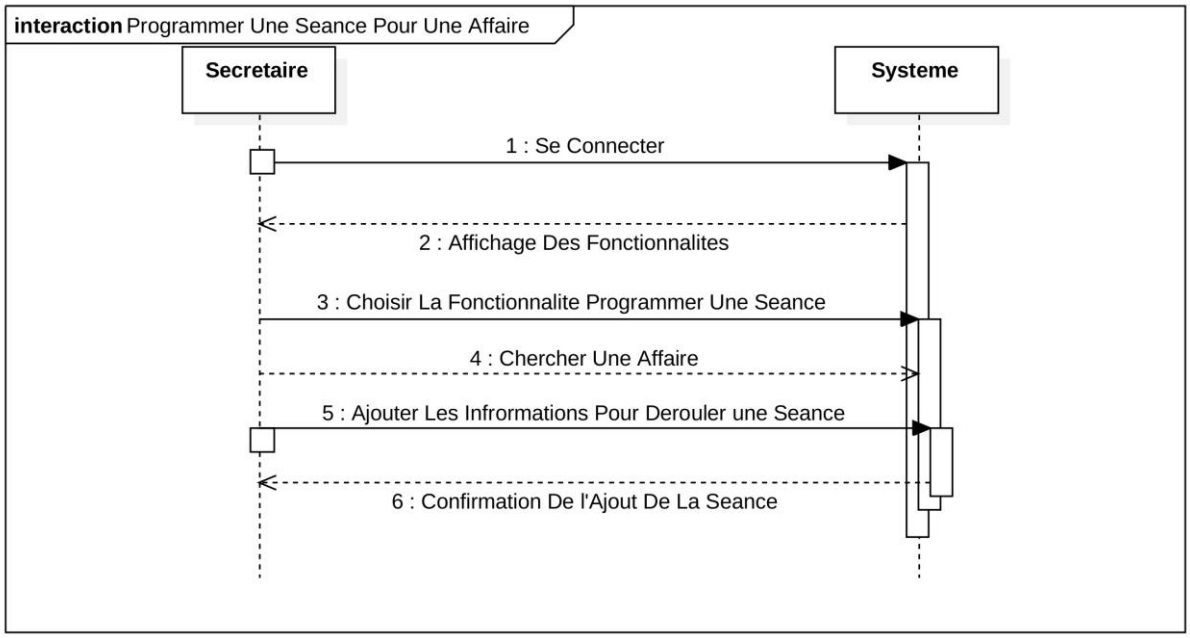

Figure 8 : Diagramme de séquence
“Programmer Une Séance Pour Un Client”

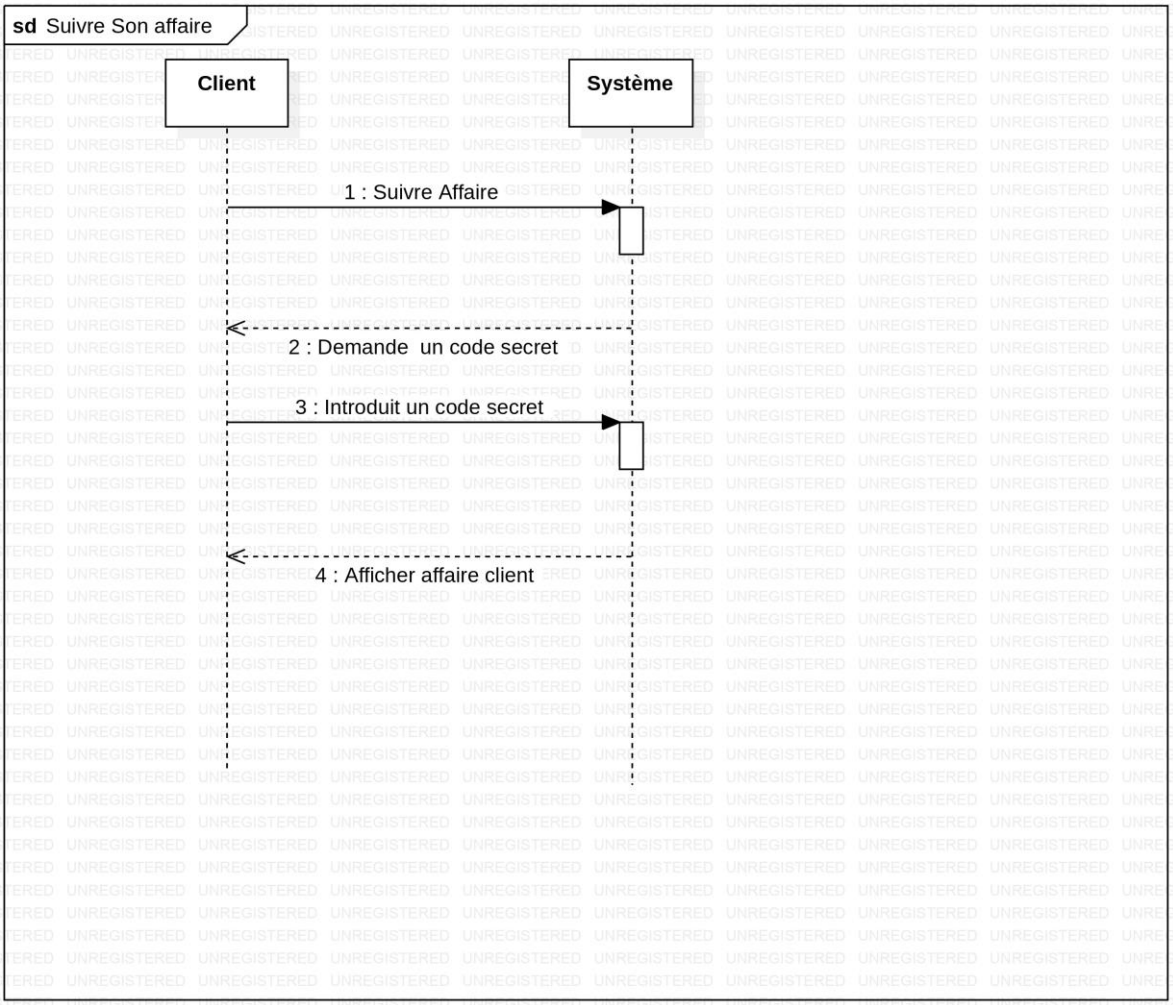

Figure 9 : Diagramme de séquence
“Suivre Son Affaire”

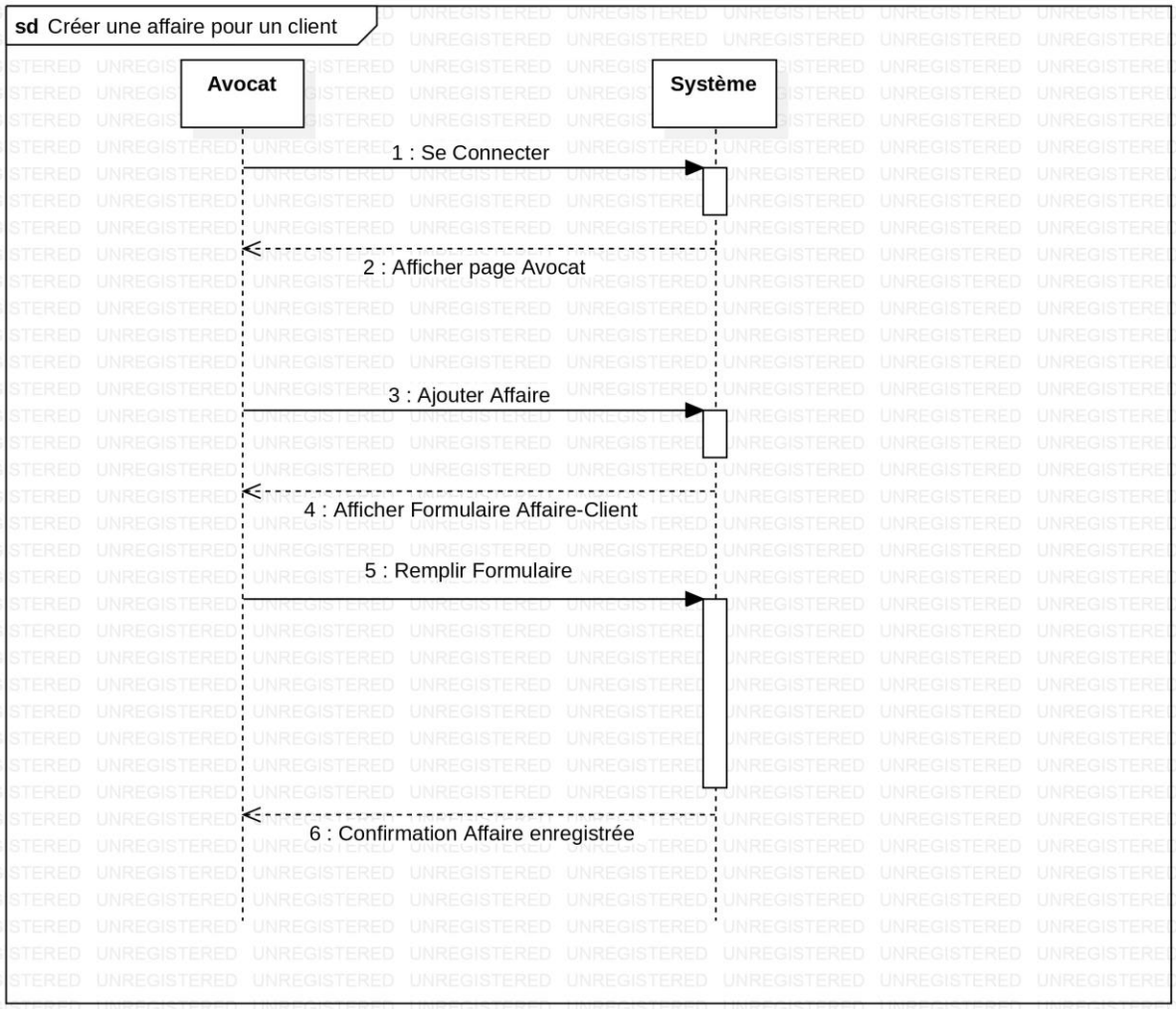

Figure 10 : Diagramme de séquence
Une Affaire Pour Un Client”

3.8 Programmation

L’application a été programmée en adoptant une architecture séparée entre le backend et le frontend.

Backend (Spring Boot)

-Le backend a été développé avec Java Spring Boot.
-Il gère toute la logique métier, la sécurité et la communication avec la base de données MySQL via WAMP.
-L’authentification utilise JWT et les mots de passe sont sécurisés avec BCrypt.

Frontend (React / JSX)

-Le frontend a été réalisé en React avec JSX pour créer une interface moderne et dynamique.
-Il communique avec l'API Spring Boot via Axios et utilise Contexte pour gérer l'état (authentification, données affaires, clients...).

Base de données (MySQL – WAMP)

-La base de données est hébergée sur WAMP, regroupant les différentes tables (avocats, juges, salles, admins...).
-Les échanges entre le backend et la base sont gérés automatiquement via Spring Data JPA.

Points forts de la programmation

-Architecture claire et bien organisée.

-Sécurité renforcée (JWT + BCrypt).

-Interface moderne et réactive.

-Système facilement extensible.

-Communication fluide entre React et Spring Boot.

3.9 Test unitaire

On testera ici chaque module séparément afin de vérifier son bon fonctionnement indépendant du reste de l'application.
Le but est de s'assurer que chaque composant remplit correctement son rôle avant son intégration.
Ainsi, on pourra détecter rapidement les erreurs locales et les corriger sans impacter l'ensemble du système.

Pour la vérification des fonctionnalités développées, nous avons utilisé l'extension Thunder Client, qui permet de tester les différentes requêtes et routes de l'application de manière simple et efficace.

4. Le script SQL de la Base De Données:

```sql
-- Database: `justice`
--

-- --------------------------------------------------------

--
-- Table structure for table `admins`
--

DROP TABLE IF EXISTS `admins`;
CREATE TABLE IF NOT EXISTS `admins` (
  `adminId` varchar(50) NOT NULL,
  `username` varchar(255) NOT NULL,
  `email` varchar(100) DEFAULT NULL,
  `password` varchar(255) DEFAULT NULL,
  `tribunalId` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`adminId`),
  UNIQUE KEY `email` (`email`),
  KEY `tribunalId` (`tribunalId`(250))
)
-- --------------------------------------------------------

--
-- Table structure for table `affaires`
--

DROP TABLE IF EXISTS `affaires`;
CREATE TABLE IF NOT EXISTS `affaires` (
  `affaireId` varchar(100) NOT NULL,
  `typeAffaire` varchar(255) DEFAULT NULL,
  `description` text,
  `dateCreation` timestamp NULL DEFAULT NULL,
  `statut` varchar(100) DEFAULT NULL,
  `tribunalId` varchar(255) DEFAULT NULL,
  `avocatId` varchar(255) DEFAULT NULL,
  `secretaireId` varchar(255) DEFAULT NULL,
```

```sql
  PRIMARY KEY (`affaireId`),
  KEY `secretaireId` (`secretaireId`(250)),
  KEY `tribunalId` (`tribunalId`(250)),
  KEY `avocatId` (`avocatId`(250))
)
-- -------------------------------------------------------


--
-- Table structure for table `avocats`
--

DROP TABLE IF EXISTS `avocats`;
CREATE TABLE IF NOT EXISTS `avocats` (
  `avocatId` varchar(50) NOT NULL,
  `nom` varchar(100) NOT NULL,
  `prenom` varchar(100) NOT NULL,
  `email` varchar(100) DEFAULT NULL,
  `password` varchar(255) DEFAULT NULL,
  `telephone` varchar(20) DEFAULT NULL,
  `adresse` text,
  `specialite` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`avocatId`),
  UNIQUE KEY `email` (`email`)
)
-- -------------------------------------------------------


--
-- Table structure for table `clients`
--

DROP TABLE IF EXISTS `clients`;
CREATE TABLE IF NOT EXISTS `clients` (
  `clientId` varchar(100) NOT NULL,
  `nom` varchar(255) NOT NULL,
  `prenom` varchar(255) NOT NULL,
  `email` varchar(255) DEFAULT NULL,
  `telephone` varchar(50) DEFAULT NULL,
  `adresse` text,
  `typeClient` varchar(100) DEFAULT NULL,
  `dateDeNaissance` date DEFAULT NULL,
  `willaya` varchar(255) DEFAULT NULL,
  `avocatId` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`clientId`),
  KEY `avocatId` (`avocatId`(250))
)
-- -------------------------------------------------------
```

```sql
--
-- Table structure for table `juges`
--

DROP TABLE IF EXISTS `juges`;
CREATE TABLE IF NOT EXISTS `juges` (
  `jugeId` varchar(50) NOT NULL,
  `password` varchar(255) DEFAULT NULL,
  `nom` varchar(255) NOT NULL,
  `prenom` varchar(255) NOT NULL,
  `email` varchar(100) DEFAULT NULL,
  `telephone` varchar(50) DEFAULT NULL,
  `specialite` varchar(255) DEFAULT NULL,
  `tribunalId` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`jugeId`),
  UNIQUE KEY `email` (`email`),
  KEY `tribunalId` (`tribunalId`(250))
)
-- --------------------------------------------------------


--
-- Table structure for table `salles`
--

DROP TABLE IF EXISTS `salles`;
CREATE TABLE IF NOT EXISTS `salles` (
  `salleId` varchar(100) NOT NULL,
  `statut` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`salleId`)
)
-- --------------------------------------------------------


--
-- Table structure for table `secretaires`
--

DROP TABLE IF EXISTS `secretaires`;
CREATE TABLE IF NOT EXISTS `secretaires` (
  `secretaireId` varchar(50) NOT NULL,
  `nom` varchar(255) NOT NULL,
  `prenom` varchar(255) NOT NULL,
  `email` varchar(100) DEFAULT NULL,
  `password` varchar(255) DEFAULT NULL,
  `telephone` varchar(50) DEFAULT NULL,
  `tribunalId` varchar(255) DEFAULT NULL,
```

```sql
  PRIMARY KEY (`secretaireId`),
  UNIQUE KEY `email` (`email`),
  KEY `tribunalId` (`tribunalId`(250))
)
-- ------------------------------------------------------


--
-- Table structure for table `tribunals`
--

DROP TABLE IF EXISTS `tribunals`;
CREATE TABLE IF NOT EXISTS `tribunals` (
  `tribunalId` varchar(50) NOT NULL,
  `nom` varchar(255) NOT NULL,
  `willaya` varchar(255) DEFAULT NULL,
  `commune` varchar(255) DEFAULT NULL,
  `adresse` varchar(255) DEFAULT NULL,
  `type` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`tribunalId`)
)
COMMIT;
```

5. Aperçu graphique de l’interface

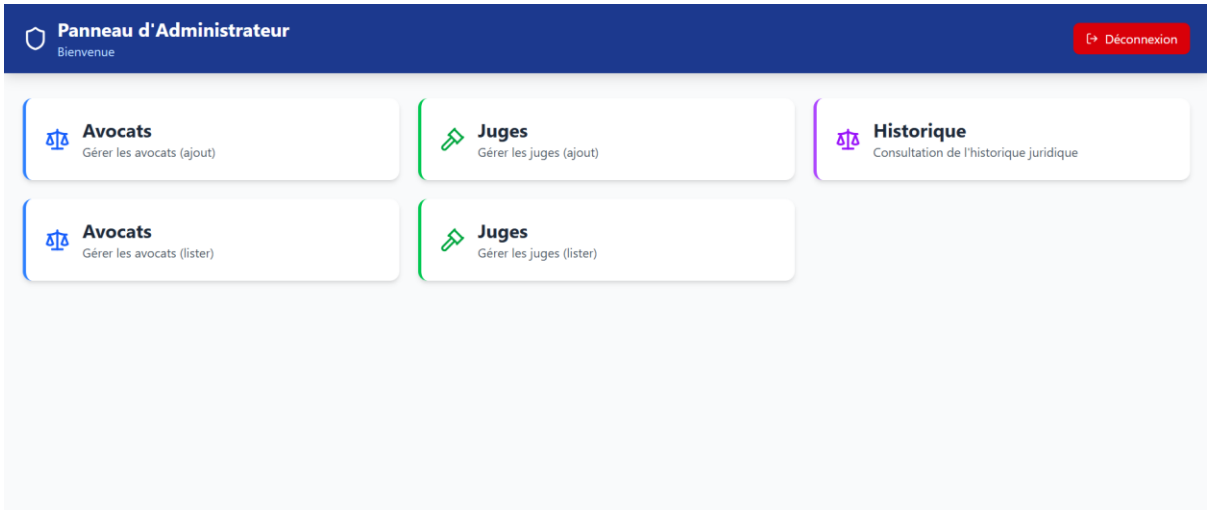

Figure 11 : Panneau d’Administrateur

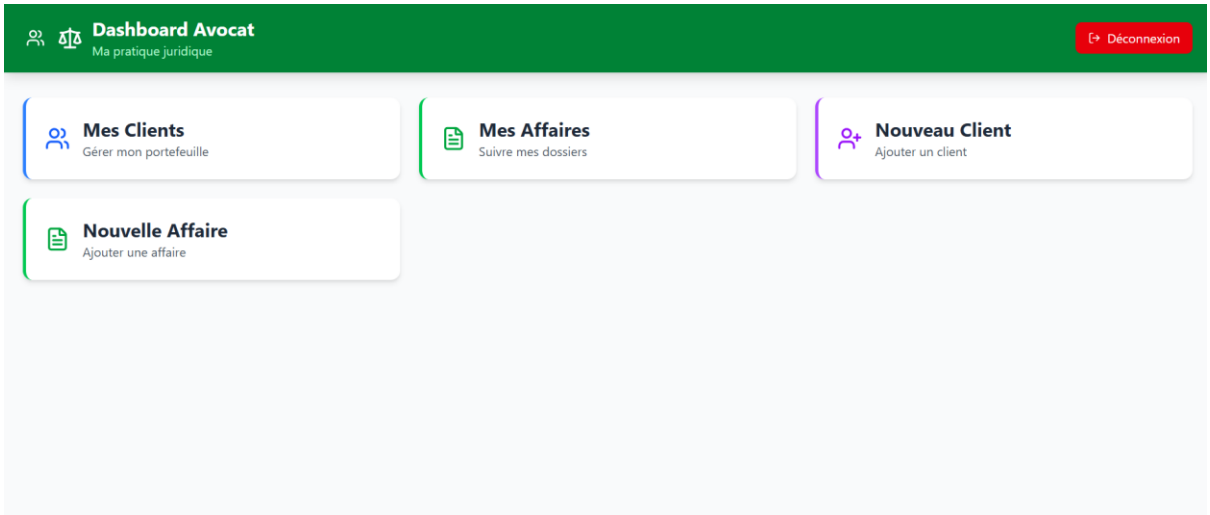

Figure 12 : Panneau d’avocat

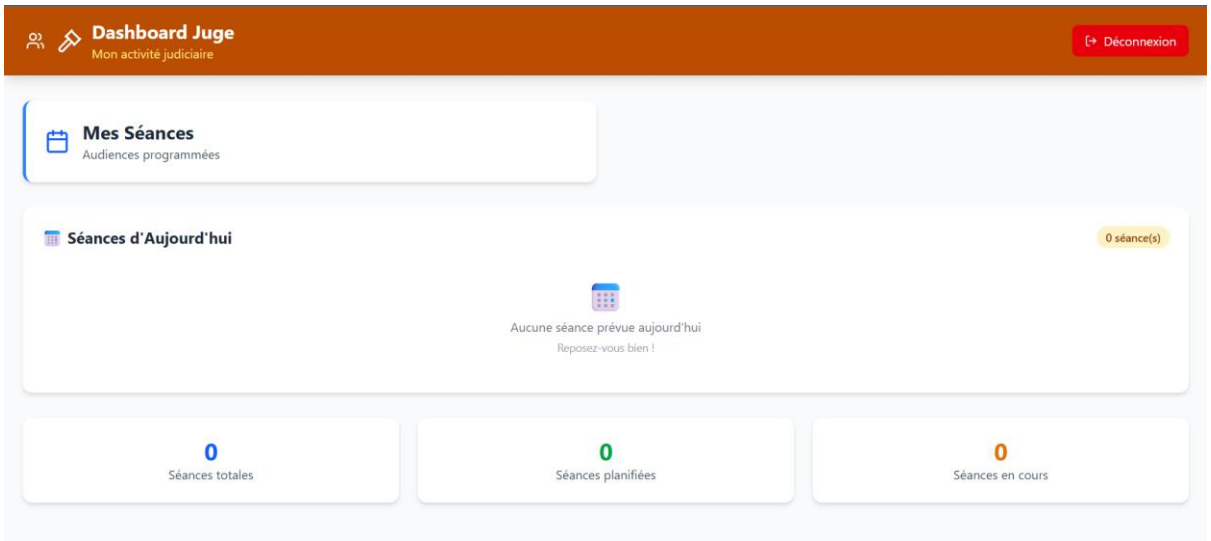

Figure 13 : Panneau du Juge

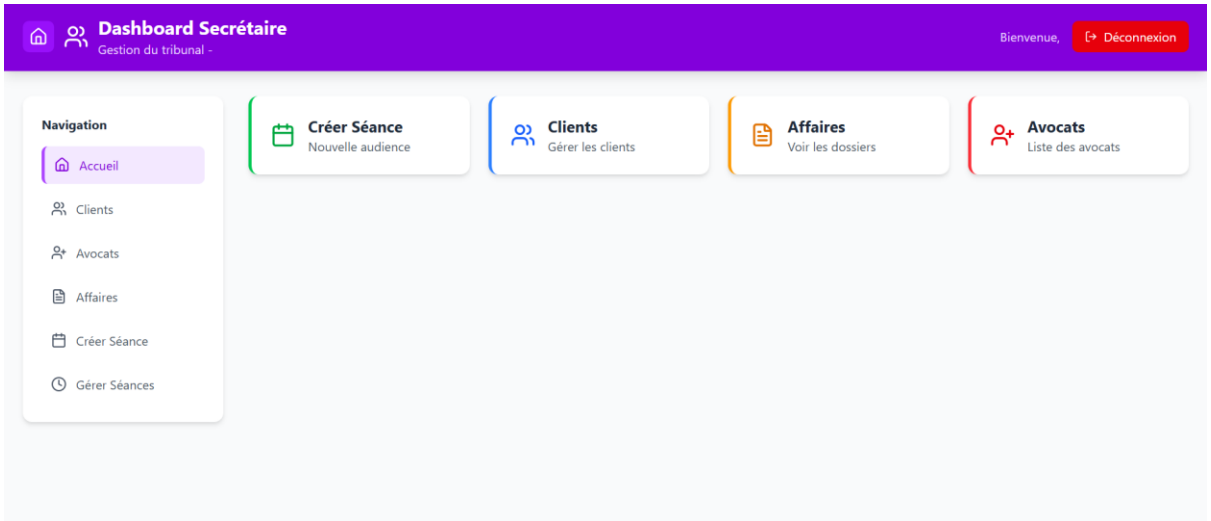

Figure 14 : Panneau du Secretaire

6. Conclusion

Ce mini-projet nous a offert l'opportunité d'appliquer de manière concrète les différentes étapes du génie logiciel, depuis l'analyse des besoins jusqu'aux tests finaux. Grâce à cette démarche structurée, nous avons pu concevoir un système fiable et cohérent répondant aux exigences d'un domaine réel : la gestion des affaires juridiques.

Le projet a permis de mettre en évidence plusieurs points forts :

-Une analyse approfondie du domaine juridique, avec une compréhension claire des relations entre clients, affaires et séances.

-Une modélisation UML complète, facilitant la communication, la cohérence et la validation des choix de conception.

-Une architecture modulaire et maintenable, rendant le système évolutif et adaptable à de futurs besoins.

-Une gestion de rôles bien définie (Admin, avocat, Secrétaire, Juge, Client), garantissant une utilisation sécurisée et organisée.

-Des fonctionnalités robustes telles que la gestion des salles, le suivi des séances, la recherche multicritères et l'authentification.

-Des tests unitaires et d'intégration, assurant la qualité et le bon fonctionnement de l'application.

En finalité, ce projet nous a permis de renforcer nos compétences techniques (UML, structuration logicielle, conception de base de données) ainsi que nos compétences méthodologiques (travail en équipe, documentation, gestion de projet).
Le système obtenu constitue une solution fonctionnelle, fiable et prête à être enrichie par de futures améliorations.